

Optical_System_for_DPD

March 16, 2026

```
[1]: import numpy as np
from scipy import signal
import tensorflow as tf
from tensorflow.keras import layers, Model, initializers, Sequential
from optic.models.devices import mzm, photodiode, edfa, iqm, coherentReceiver,
    ↪pdmCoherentReceiver, basicLaserModel
from optic.models.channels import linearFiberChannel, ssfm
from optic.comm.modulation import modulateGray, grayMapping
from optic.comm.sources import bitSource, symbolSource
from optic.dsp.core import upsample, pulseShape, pnorm, anorm, signalPower,
    ↪firFilter, decimate, symbolSync, phaseNoise

try:
    from optic.dsp.coreGPU import checkGPU
    if checkGPU():
        from optic.dsp.coreGPU import firFilter
    else:
        from optic.dsp.core import firFilter
except ImportError:
    from optic.dsp.core import firFilter

from optic.utils import parameters, dBm2W, ber2Qfactor
from optic.plot import eyediagram, pconst, plotPSD
import matplotlib.pyplot as plt
from scipy.special import erfc
from tqdm.notebook import tqdm
import scipy as sp
import scipy.constants as const

try:
    from optic.models.modelsGPU import manakovSSF
except:
    from optic.models.channels import manakovSSF

from optic.dsp.equalization import edc, mimoAdaptEqualizer, ffe
from optic.dsp.carrierRecovery import cpr
```

```

from optic.comm.metrics import fastBERcalc, monteCarloGMI, monteCarloMI,
    ↪ calcEVM, bert
from optic.dsp.clockRecovery import gardnerClockRecovery

import logging as logg
logg.basicConfig(level=logg.INFO, format='%(message)s', force=True)
import time

```

```

[2]: from IPython.core.display import HTML
from IPython.core.pylabtools import figsize

HTML("""
<style>
.output_png {
    display: table-cell;
    text-align: center;
    vertical-align: middle;
}
</style>
""")

```

[2]: <IPython.core.display.HTML object>

```

[3]: #_
    ↪ -----
# Power Amplifier (PA) Functions
#_
    ↪ -----

def rapp_pa(x, Vsat, p=2.0):
    """
    Memoryless Rapp PA model for complex baseband input.

    Parameters
    -----
    x : np.ndarray
        Complex input waveform.
    Vsat : float
        Saturation voltage/amplitude.
    p : float
        Rapp smoothness exponent. Paper uses p=2.

    Returns
    -----
    y : np.ndarray
        Output after memoryless PA nonlinearity.
    """

```

```

"""

mag = np.abs(x)
gain = 1.0 / (1.0 + (mag / Vsat) ** (2 * p)) ** (1.0 / (2 * p))
return x * gain

def butter_lpf_complex(x, Fs, f3dB, order=3):
    """
    Apply an order-N Butterworth LPF to a complex baseband waveform.
    """

    wn = f3dB / (Fs / 2) # normalized cutoff
    if wn >= 1.0:
        raise ValueError(
            f"Butterworth cutoff must be below Nyquist. Got f3dB={f3dB/1e9:.2f} GHz, "
            f"Fs/2={(Fs/2)/1e9:.2f} GHz."
        )

    # This function creates the coefficients of the Butterworth filter
    # b = numerator coefficient
    # a = denominator coefficients
    b, a = signal.butter(order, wn, btype='low')

    # Apply the filter at the real and complex parts separately
    y_i = signal.lfilter(b, a, np.real(x))
    y_q = signal.lfilter(b, a, np.imag(x))

    # Recombine the signal
    return y_i + 1j * y_q

def pa_model(x, Fs, BO_dB=5.0, p=2.0, f3dB=37e9):
    """
    Paper-style PA model:
    x -> Rapp nonlinearity -> 3rd-order Butterworth LPF

    Parameters
    -----
    x : np.ndarray
        Complex baseband input waveform.
    Fs : float
        Sample rate [Hz].
    BO_dB : float
        Back-off setting in dB. Use 7, 5, 3 for weak/moderate/strong.
    p : float

```

```

    Rapp exponent. Paper uses p=2.
    f3dB : float
    PA 3-dB bandwidth [Hz]. Paper uses 37 GHz.

Returns
-----
y : np.ndarray
    PA output waveform.
info : dict
    Diagnostic information.
"""

# Calculate the RMS voltage of the electrical complex waveform (sigTx)
Vrms = np.sqrt(np.mean(np.abs(x) ** 2))

# Use back-off definition so smaller BO => stronger nonlinearity
# Vsat larger than Vrms for weakly nonlinear operation.
Vsat = Vrms * 10 ** (BO_dB / 20)

y_nl = rapp_pa(x, Vsat=Vsat, p=p)
y = butter_lpf_complex(y_nl, Fs=Fs, f3dB=f3dB, order=3)

# Define PA info
info = {
    "Vrms_in": Vrms,
    "Vsat": Vsat,
    "BO_dB": BO_dB,
    "p": p,
    "f3dB_Hz": f3dB
}
return y, info

```

```

[4]: #_
↳ -----
# Parameters Configuration of Optical System
#_
↳ -----

# -----
# Transmitter Parameters
#-----

# 1) General Parameters
SpS = 16          # samples per symbol
Rs = 32e9         # symbol rate [baud]
Fs = Rs * SpS     # sampling frequency [Hz]

```

```

M = 16                                # QPSK -> M=4, constType='qam'
nBits = 400000                        # total bits to generate
rollOff = 0.01                        # RRC roll-off
nFilterTaps = 1024                    # RRC filter taps
mzmScale = 1.5 #0.5                   # IQM drive scale = "modulation depth" (0.5 is
    ↪ default in OptiCommPy)
Vpi = 2                              # IQM pi Voltage (2V is default in OptiCommPy)
P_launch_dBm = 0                      # desired launched optical power [dBm] -- for 4QAM try
    ↪ 6 dbm -- for 16QAM 0 was good
laserLinewidth = 100e3                # Hz (set e.g. 100e3 for phase noise), setting
    ↪ laserLineWidth to 0, models the ideal case

# 2) Symbol Source Parameters
paramSymb = parameters()
paramSymb.nSymbols = int(nBits // np.log2(M)) # symbols = bits / log2(M)
paramSymb.M = M
paramSymb.constType = "qam"           # 'qam' with M=4 -> QPSK
paramSymb.dist = "uniform"            # uniform symbol probabilities
paramSymb.seed = 444
paramSymb.shapingFactor = 0

constSymb = grayMapping(paramSymb.M, paramSymb.constType)
if paramSymb.dist == "uniform":
    px = np.ones(paramSymb.M) / paramSymb.M
elif paramSymb.probDist == "maxwell-boltzmann":
    px = np.exp(-paramSymb.shapingFactor * np.abs(constSymb) ** 2)
    px = px / np.sum(px)
else:
    raise ValueError("Invalid probability distribution.")
paramSymb.px = px

# 3) UpSampling and FIR Parameters
paramPulse = parameters()
paramPulse.pulseType = "rrc"
paramPulse.nFilterTaps = nFilterTaps
paramPulse.rollOff = rollOff
paramPulse.SpS = SpS

# 4) IQM Parameters
paramIQM = parameters()
paramIQM.Vpi = Vpi
paramIQM.VbI = -Vpi
paramIQM.VbQ = -Vpi
paramIQM.Vphi = Vpi/2

# 5) Optical Carrier / LO field (Ein)
sigTx_length = paramSymb.nSymbols * SpS

```

```

if laserLinewidth and laserLinewidth > 0:
    phi_pn = phaseNoise(laserLinewidth, sigTx_length, 1 / Fs, seed=123)
    sigL0 = np.exp(1j * phi_pn)
else:
    sigL0 = np.ones_like(sigTx, dtype=complex)

# 6) PA parameters
PA_enable = False      # enable flag. If "False", the PA is skipped
PA_BO_dB = 3           # 7 = weak, 5 = moderate, 3 = strong
PA_p = 2.0             # paper says typically p=2
PA_f3dB = 18.5e9       # frequency of 3 dB

# -----
# Channel Parameters
#-----

# 1) Optical Channel Parameters
paramCh = parameters()
paramCh.Ltotal = 80      # total link distance [km]
paramCh.Lspan = 80       # span length [km]
paramCh.alpha = 0.2      # fiber loss parameter [dB/km]
paramCh.D = 16           # fiber dispersion parameter [ps/nm/km]
paramCh.gamma = 1.3      # fiber nonlinear parameter [1/(W.km)]
paramCh.Fc = 193.1e12    # central optical frequency of the WDM spectrum
paramCh.hz = 0.5         # step-size of the split-step Fourier method [km]
paramCh.prgsBar = True   # show progress bar
paramCh.Fs = Rs*SpS      # sampling rate
paramCh.amp = 'edfa'     # type of amplifiers
paramCh.NF = 4.5         # Noise Figure of each amplifier [dB]
#paramCh.seed = 456

# -----
# Receiver Parameters
#-----

# 1) local oscillator (LO) parameters:
FO = -128e6              # frequency offset
paramLO = parameters()
paramLO.P = 2            # power in dBm
paramLO.lw = 100e3       # laser linewidth
paramLO.RIN_var = 0
paramLO.Fs = Fs
paramLO.seed = 789 # random seed for noise generation
paramLO.freqShift = 0 + FO # downshift of the channel to be demodulated add_
↪ frequency offset

```

```
# 2) Front-End Parameters and photodiode paramters
```

```
# Frontend parameters
```

```
paramFE = parameters()
```

```
paramFE.Fs = Fs
```

```
# Photodiodes parameters
```

```
paramPD = parameters()
```

```
paramPD.B = Rs
```

```
paramPD.Fs = Fs
```

```
paramPD.ideal = True
```

```
paramPD.seed = 1011
```

```
# 3) Pulseshaping in the reciever using rrc filter
```

```
paramRxPulse = parameters()
```

```
paramRxPulse.SpS = SpS
```

```
paramRxPulse.nFilterTaps = nFilterTaps
```

```
paramRxPulse.rollOff = rollOff
```

```
paramRxPulse.pulseType = "rrc"
```

```
# 4) Decimation Parameters
```

```
paramDec = parameters()
```

```
paramDec.SpSin = SpS
```

```
paramDec.SpSout = 2
```

```
# 5) Chromatic Dispersion Parameters
```

```
paramEDC = parameters()
```

```
paramEDC.L = paramCh.Ltotal
```

```
paramEDC.D = paramCh.D
```

```
paramEDC.Fc = paramCh.Fc
```

```
paramEDC.Rs = Rs
```

```
paramEDC.Fs = 2*Rs
```

```
# 6) Adaptive Equalization Parameters
```

```
paramEq = parameters()
```

```
paramEq.nTaps = 35
```

```
paramEq.SpS = paramDec.SpSout
```

```
paramEq.numIter = 2
```

```
paramEq.storeCoeff = False
```

```
paramEq.M = M
```

```
paramEq.shapingFactor = 0
```

```
paramEq.constType = "qam"
```

```
paramEq.prgsBar = False
```

```
# 7) Data-Aided or Blind Reciever Equalization
```

```
# Can be set here or not
```

```

#Data_Aided = True

# 8) Carrier and Phase recovery parameters using bps
paramCPR = parameters()
paramCPR.alg = 'bps'
paramCPR.M = M
paramCPR.constType = "qam"
paramCPR.shapingFactor = 0
paramCPR.N = 25
paramCPR.B = 64
paramCPR.returnPhases = True
paramCPR.Ts = 1/Rs

```

```

[5]: # -----
# Simulation of the Optical System
# -----

def simulate_optical_system(symbTx, paramSymb, paramPulse,paramIQM,sigL0,
    ↪paramCh, paramL0, paramFE, paramPD,paramRxPulse,paramDec
    ,paramEDC,paramEq,paramCPR,
    ↪PA_enable=False,Data_Aided=True):

    """
    This function simulates the optical transmission of a single-polarization,
    single-channel 16/4-QAM signal.

    Parameters
    -----
    Uses Global Parameters defined above

    Returns
    -----
    y_CPR_1 :
        Received signal at end of DSP chain (after CPR)
    d:
        Noemlaized Symbol Synchronization with the SymbTx. Used in calculations.
    """

    #_
    ↪-----
    # TRANSMITTER

    # 2) Perform Upsampling + Pulse Shaping (RRC)
    pulse = pulseShape(paramPulse)          # create pulse
    symbolsUp = upsample(symbTx, SpS)        # shape: (nSymbols*SpS,)
    sigTx = firFilter(pulse, symbolsUp)      # pulse-shaped baseband waveform

```



```

    sigTx = sigTx / np.max(np.abs(sigTx))    # normalize waveform to peak
    ↪ amplitude 1 (NOT RMS). This makes drive scaling predictable.

    # 3) Power Amplifier PA & 3rd-Order Butterworth LPF
    if PA_enable:
        sigTx_PA, paInfo = pa_model(sigTx, Fs=Fs, BO_dB=PA_BO_dB, p=PA_p,
    ↪ f3dB=PA_f3dB)
    else:
        sigTx_PA = sigTx.copy()

    # 4) IQ Modulation: Electrical -> Optical Domain Conversion
    # u_drive = mzmScale * sigTx_PA
    u_drive = (Vpi* mzmScale * sigTx_PA)

    sigTxo = iqm(sigLO, u_drive, paramIQM)    # complex optical field at IQM
    ↪ output (still normalized power)

    # 5) Set Launched Optical Power
    P_launch_W = dBm2W(P_launch_dBm)
    sigTxo = np.sqrt(P_launch_W) * pnorm(sigTxo)

    # End of Transmitter
    #
    ↪ -----

    #
    ↪ -----

    # CHANNEL

    sigCh = ssfm(sigTxo, paramCh)

    # End of CHANNEL
    #
    ↪ -----

    #
    ↪ -----

    # RECEIVER

    # 1) Generate CW laser LO field
    # Parameter which might change so needs to be in the function
    paramLO.Ns = len(sigCh)
    sigLO_Rx = basicLaserModel(paramLO)

```

```

# 2) Coherent Reciever for single-polarization.
# Contains a 90 hybrid, two balanced photodiodes and iqmixing
sigRxFrontEnd = coherentReceiver(sigCh, sigLO_Rx, paramFE, paramPD)

# 3) Puleshaping Function
pulse = pulseShape(paramRxPulse)
sigRxPulseShape = firFilter(pulse, sigRxFrontEnd)

# 4) Decimation
sigRxDecimation = decimate(sigRxPulseShape, paramDec)

# 5) Chromatic Dispersion Compensation for non-linear channel
sigRxCD = edc(sigRxDecimation, paramEDC)

# 6) Symbol Synchronization with the SymbTx
symbRxCD = symbolSync(sigRxCD, symbTx, 2)

# 7) Power Normalization
x = pnorm(sigRxCD)
d = pnorm(symbRxCD)

# 8) Adaptive Equalization Parameter which might change
paramEq.L = [int(0.2*d.shape[0]), int(0.8*d.shape[0])]

# 9) Data Aided or Blind
if Data_Aided:
    if M == 4:
        paramEq.alg = ['cma', 'cma'] # QPSK
        paramEq.mu = [5e-3, 1e-3]
        y_EQ = mimoAdaptEqualizer(x, paramEq, d)
    else:
        paramEq.alg = ['da-rde', 'rde'] # M-QAM
        paramEq.mu = [5e-3, 5e-4]
        y_EQ = mimoAdaptEqualizer(x, paramEq, d)
else:
    if M == 4:
        paramEq.alg = ['cma', 'cma'] # QPSK
        paramEq.mu = [5e-3, 1e-3]
        y_EQ = mimoAdaptEqualizer(x, paramEq, None)
    else:
        paramEq.alg = ['cma', 'rde'] # M-QAM
        paramEq.mu = [5e-3, 1e-3]
        y_EQ = mimoAdaptEqualizer(x, paramEq, None)

# 9) Carrier and Phase recovery using bps in the parameters
y_CPR_1, = cpr(y_EQ, paramCPR)

```

```

return y_CPR_1,d
# End of RECEIVER
#

```

```

[6]: def perf_calc(symbTx, y_CPR_1, d, M, paramSymb, paramDec):

    """Performance Metric Exploration for Single Polarization"""

    # remove phase ambiguity for 4-QAM/QPSK
    if M == 16:
        discard = 5000
        ind = np.arange(discard, len(symbTx) - discard)

        d = symbTx

    # now compute metrics (both are 1 sample/symbol and same length)
    BER, SER, SNR = fastBERcalc(y_CPR_1[ind], d[ind], M, 'qam', px =
    paramSymb.px)
    EVM = calcEVM(y_CPR_1[ind], M, 'qam', d[ind])
    Qfactor = ber2Qfactor(BER[0])

    print(' SER: %.3e,  '%(SER[0]))
    print(' BER: %.3e  '%(BER[0]))
    print(' SNR: %.3f dB'%(SNR[0]))
    print(' EVM: %.3f %'%(EVM[0]*100))
    print(' Qfactor: %.3f,  '%(Qfactor))

    return BER

```

```

[7]: symbTx = symbolSource(paramSymb)

# Running the function to check that it works
y_CPR_1, d= simulate_optical_system(symbTx, paramSymb,
    paramPulse,paramIQM,sigLO, paramCh, paramLO, paramFE,
    paramPD,paramRxPulse,paramDec
    ,paramEDC,paramEq,paramCPR, PA_enable=False,
    Data_Aided=True)

perf_calc(symbTx, y_CPR_1, d, M, paramSymb, paramDec)

discard = 5000
# plot constellations

```

```

pconst(y_CPR_1[discard:-discard])

# plotting eye diagrams of sigTx
eyediagram(y_CPR_1.real[discard:-discard], y_CPR_1.real.size-2*discard,
↳paramDec.SpSout, plotlabel='signal at Rx REAL', ptype='fancy')
eyediagram(y_CPR_1.imag[discard:-discard], y_CPR_1.imag.size-2*discard,
↳paramDec.SpSout, plotlabel='signal at Rx IMAGNIARY', ptype='fancy')

```

```

0%|          | 0/1 [00:00<?, ?it/s]

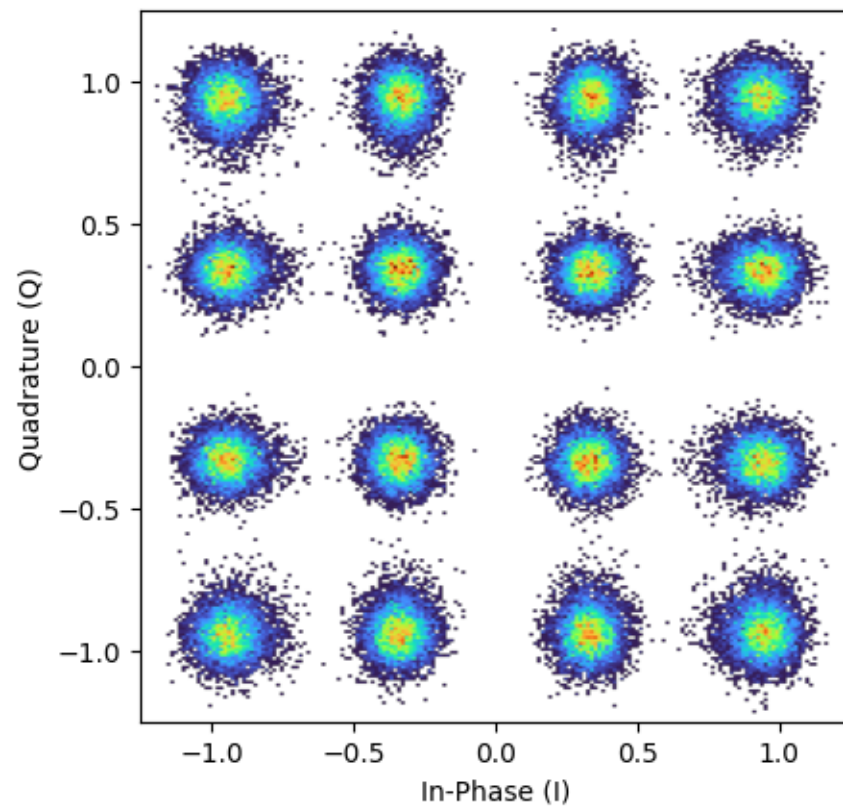
```

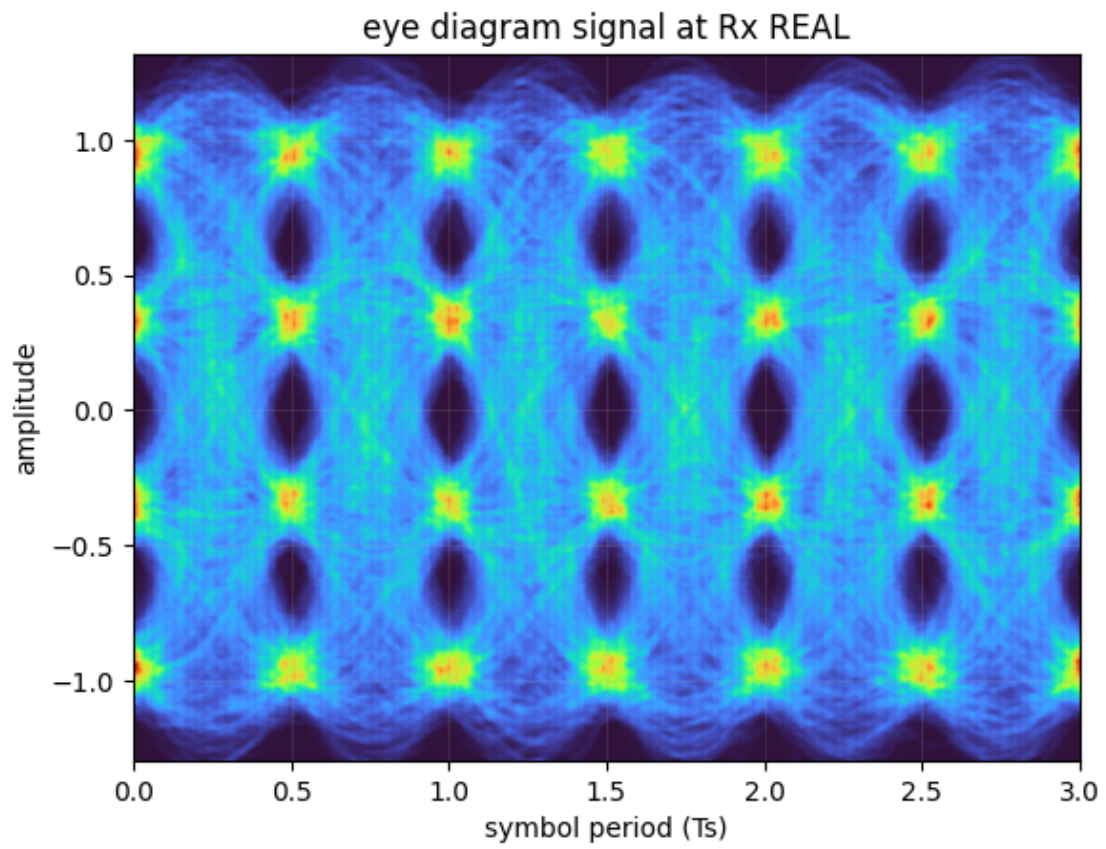
```

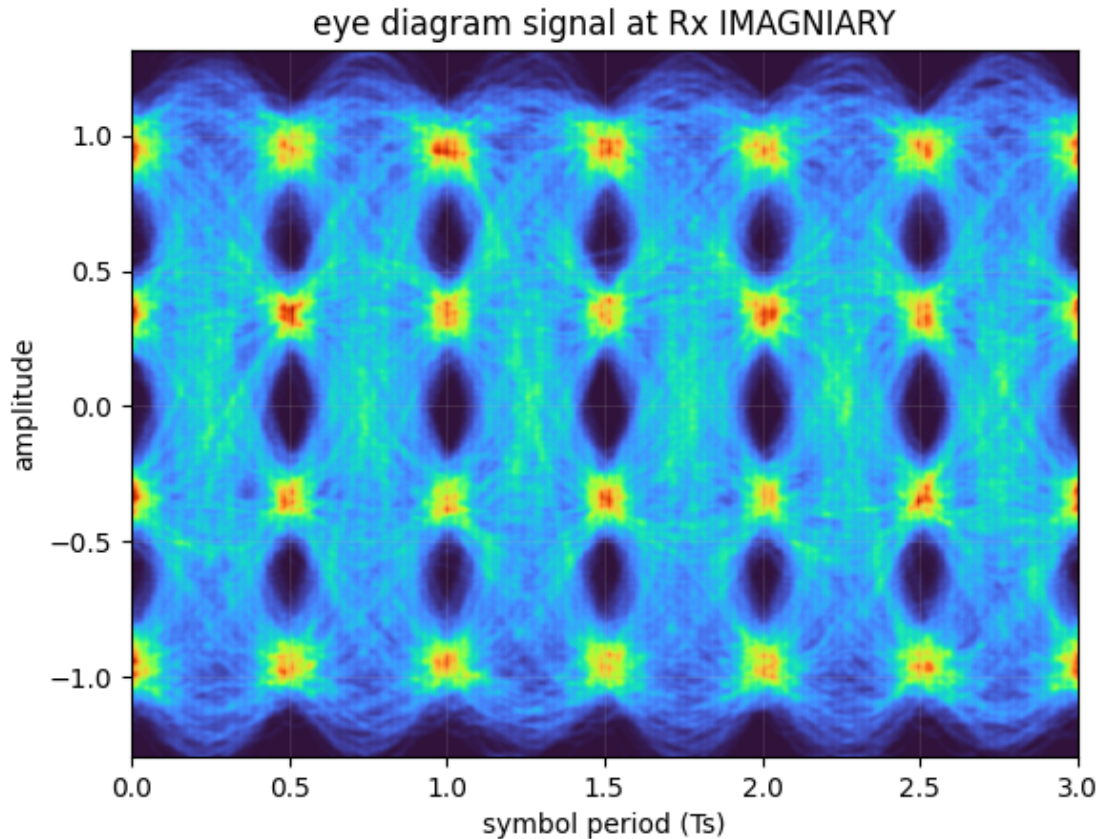
Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.030180.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.024184.
rde - training stage #1
rde MSE = 0.018581.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 398.637 kHz

SER: 5.000e-04,
BER: 1.250e-04
SNR: 20.275 dB
EVM: 0.938 %
Qfactor: 5.637,

```







0.1 DPD

```
[8]: def build_model():
    inputs = layers.Input(shape=(None, 2)) # 2 for I and Q

    sec_a = layers.Conv1D(2, 100, padding='same')(inputs) # 100 taps was a
    ↪sweet spot, 20-ish fails to converge, tiker with different values.

    # sec_a = inputs

    nonlinear_1 = layers.Dense(20, activation=layers.LeakyReLU(0.1))(sec_a)
    nonlinear_2 = layers.Dense(20, activation=layers.LeakyReLU(0.
    ↪1))(nonlinear_1)
    nonlinear_3 = layers.Dense(2, activation=layers.LeakyReLU(0.1))(nonlinear_2)
    nonlinear_4 = layers.Dense(2, activation='linear')(nonlinear_3)

    #outputs = layers.Add()([sec_a, nonlinear_4]) # PLEASE IGNORE
    outputs = nonlinear_4

    return Model(inputs, outputs)
```

```
seq_length = 5000
```

```
# ILA
```

```
dpd_model = build_model()
```

```
dpd_model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-2),  
↳loss='mse') # ILA
```

```
2026-03-16 19:28:10.611115: I metal_plugin/src/device/metal_device.cc:1154]  
Metal device set to: Apple M3  
2026-03-16 19:28:10.611152: I metal_plugin/src/device/metal_device.cc:296]  
systemMemory: 24.00 GB  
2026-03-16 19:28:10.611163: I metal_plugin/src/device/metal_device.cc:313]  
maxCacheSize: 8.88 GB  
2026-03-16 19:28:10.611177: I  
tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:305]  
Could not identify NUMA node of platform GPU ID 0, defaulting to 0. Your kernel  
may not have been built with NUMA support.  
2026-03-16 19:28:10.611185: I  
tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:271]  
Created TensorFlow device (/job:localhost/replica:0/task:0/device:GPU:0 with 0  
MB memory) -> physical PluggableDevice (device: 0, name: METAL, pci bus id:  
<undefined>)
```

```
[9]: def split_i_q(arr):  
    return np.stack([np.real(arr), np.imag(arr)]).T  
  
def merge_i_q(arr):  
    if arr.ndim == 3:  
        return arr[:, :, 0] + 1j*arr[:, :, 1]  
    elif arr.ndim == 2:  
        return arr[:, 0] + 1j*arr[:, 1]  
    else:  
        raise ValueError("Input array must be 2D or 3D.")
```

```
[10]: reshabe_len = len(symbTx)//seq_length  
  
symbTx_nn = split_i_q(symbTx)  
symbTx_nn = symbTx_nn[:reshabe_len*seq_length].reshape(-1, seq_length, 2) #  
↳batching the symbols for training (shape: num_batches, seq_length,  
↳num_features)  
  
symbTx_nn.shape
```

```
[10]: (20, 5000, 2)
```



```
[11]: dpd_model.fit(symbTx_nn, symbTx_nn, epochs=350, verbose=1) # this line is ↵  
      ↪important, = starting as a passthrough.
```

Epoch 1/350

2026-03-16 19:28:11.074046: I

tensorflow/core/grappler/optimizers/custom_graph_optimizer_registry.cc:117]

Plugin optimizer for device_type GPU is enabled.

1/1 1s 948ms/step - loss:

0.5076

Epoch 2/350

1/1 0s 32ms/step - loss:

0.4965

Epoch 3/350

1/1 0s 24ms/step - loss:

0.4942

Epoch 4/350

1/1 0s 25ms/step - loss:

0.4902

Epoch 5/350

1/1 0s 25ms/step - loss:

0.4836

Epoch 6/350

1/1 0s 24ms/step - loss:

0.4743

Epoch 7/350

1/1 0s 25ms/step - loss:

0.4611

Epoch 8/350

1/1 0s 27ms/step - loss:

0.4432

Epoch 9/350

1/1 0s 29ms/step - loss:

0.4222

Epoch 10/350

1/1 0s 27ms/step - loss:

0.4006

Epoch 11/350

1/1 0s 26ms/step - loss:

0.3815

Epoch 12/350

1/1 0s 27ms/step - loss:

0.3661

Epoch 13/350

1/1 0s 29ms/step - loss:

0.3543

Epoch 14/350

1/1 0s 30ms/step - loss:

```

0.3447
Epoch 15/350
1/1      0s 29ms/step - loss:
0.3367
Epoch 16/350
1/1      0s 27ms/step - loss:
0.3305
Epoch 17/350
1/1      0s 24ms/step - loss:
0.3246
Epoch 18/350
1/1      0s 24ms/step - loss:
0.3169
Epoch 19/350
1/1      0s 24ms/step - loss:
0.3082
Epoch 20/350
1/1      0s 24ms/step - loss:
0.3003
Epoch 21/350
1/1      0s 24ms/step - loss:
0.2943
Epoch 22/350
1/1      0s 26ms/step - loss:
0.2895
Epoch 23/350
1/1      0s 26ms/step - loss:
0.2851
Epoch 24/350
1/1      0s 26ms/step - loss:
0.2810
Epoch 25/350
1/1      0s 28ms/step - loss:
0.2776
Epoch 26/350
1/1      0s 29ms/step - loss:
0.2748
Epoch 27/350
1/1      0s 28ms/step - loss:
0.2726
Epoch 28/350
1/1      0s 25ms/step - loss:
0.2711
Epoch 29/350
1/1      0s 27ms/step - loss:
0.2701
Epoch 30/350
1/1      0s 29ms/step - loss:

```

0.2693
 Epoch 31/350
 1/1 0s 30ms/step - loss:
 0.2683
 Epoch 32/350
 1/1 0s 28ms/step - loss:
 0.2673
 Epoch 33/350
 1/1 0s 26ms/step - loss:
 0.2665
 Epoch 34/350
 1/1 0s 58ms/step - loss:
 0.2658
 Epoch 35/350
 1/1 0s 27ms/step - loss:
 0.2652
 Epoch 36/350
 1/1 0s 29ms/step - loss:
 0.2648
 Epoch 37/350
 1/1 0s 30ms/step - loss:
 0.2644
 Epoch 38/350
 1/1 0s 29ms/step - loss:
 0.2638
 Epoch 39/350
 1/1 0s 30ms/step - loss:
 0.2632
 Epoch 40/350
 1/1 0s 30ms/step - loss:
 0.2625
 Epoch 41/350
 1/1 0s 27ms/step - loss:
 0.2616
 Epoch 42/350
 1/1 0s 23ms/step - loss:
 0.2607
 Epoch 43/350
 1/1 0s 24ms/step - loss:
 0.2598
 Epoch 44/350
 1/1 0s 24ms/step - loss:
 0.2590
 Epoch 45/350
 1/1 0s 24ms/step - loss:
 0.2583
 Epoch 46/350
 1/1 0s 25ms/step - loss:

0.2578
 Epoch 47/350
 1/1 0s 28ms/step - loss:
 0.2573
 Epoch 48/350
 1/1 0s 29ms/step - loss:
 0.2569
 Epoch 49/350
 1/1 0s 22ms/step - loss:
 0.2565
 Epoch 50/350
 1/1 0s 30ms/step - loss:
 0.2561
 Epoch 51/350
 1/1 0s 27ms/step - loss:
 0.2558
 Epoch 52/350
 1/1 0s 23ms/step - loss:
 0.2555
 Epoch 53/350
 1/1 0s 24ms/step - loss:
 0.2552
 Epoch 54/350
 1/1 0s 24ms/step - loss:
 0.2549
 Epoch 55/350
 1/1 0s 24ms/step - loss:
 0.2545
 Epoch 56/350
 1/1 0s 25ms/step - loss:
 0.2542
 Epoch 57/350
 1/1 0s 28ms/step - loss:
 0.2538
 Epoch 58/350
 1/1 0s 30ms/step - loss:
 0.2535
 Epoch 59/350
 1/1 0s 23ms/step - loss:
 0.2531
 Epoch 60/350
 1/1 0s 29ms/step - loss:
 0.2527
 Epoch 61/350
 1/1 0s 26ms/step - loss:
 0.2524
 Epoch 62/350
 1/1 0s 23ms/step - loss:

```

0.2520
Epoch 63/350
1/1      0s 24ms/step - loss:
0.2515
Epoch 64/350
1/1      0s 23ms/step - loss:
0.2511
Epoch 65/350
1/1      0s 24ms/step - loss:
0.2506
Epoch 66/350
1/1      0s 25ms/step - loss:
0.2501
Epoch 67/350
1/1      0s 27ms/step - loss:
0.2495
Epoch 68/350
1/1      0s 29ms/step - loss:
0.2489
Epoch 69/350
1/1      0s 59ms/step - loss:
0.2481
Epoch 70/350
1/1      0s 28ms/step - loss:
0.2472
Epoch 71/350
1/1      0s 24ms/step - loss:
0.2462
Epoch 72/350
1/1      0s 23ms/step - loss:
0.2449
Epoch 73/350
1/1      0s 24ms/step - loss:
0.2434
Epoch 74/350
1/1      0s 24ms/step - loss:
0.2415
Epoch 75/350
1/1      0s 25ms/step - loss:
0.2391
Epoch 76/350
1/1      0s 25ms/step - loss:
0.2360
Epoch 77/350
1/1      0s 25ms/step - loss:
0.2320
Epoch 78/350
1/1      0s 27ms/step - loss:

```

```

0.2266
Epoch 79/350
1/1      0s 26ms/step - loss:
0.2191
Epoch 80/350
1/1      0s 25ms/step - loss:
0.2085
Epoch 81/350
1/1      0s 28ms/step - loss:
0.1935
Epoch 82/350
1/1      0s 28ms/step - loss:
0.1730
Epoch 83/350
1/1      0s 26ms/step - loss:
0.1483
Epoch 84/350
1/1      0s 27ms/step - loss:
0.1241
Epoch 85/350
1/1      0s 29ms/step - loss:
0.1074
Epoch 86/350
1/1      0s 22ms/step - loss:
0.0988
Epoch 87/350
1/1      0s 26ms/step - loss:
0.0930
Epoch 88/350
1/1      0s 23ms/step - loss:
0.0851
Epoch 89/350
1/1      0s 24ms/step - loss:
0.0775
Epoch 90/350
1/1      0s 24ms/step - loss:
0.0730
Epoch 91/350
1/1      0s 25ms/step - loss:
0.0696
Epoch 92/350
1/1      0s 25ms/step - loss:
0.0649
Epoch 93/350
1/1      0s 28ms/step - loss:
0.0590
Epoch 94/350
1/1      0s 30ms/step - loss:

```

```

0.0519
Epoch 95/350
1/1      0s 23ms/step - loss:
0.0448
Epoch 96/350
1/1      0s 29ms/step - loss:
0.0403
Epoch 97/350
1/1      0s 24ms/step - loss:
0.0366
Epoch 98/350
1/1      0s 23ms/step - loss:
0.0330
Epoch 99/350
1/1      0s 25ms/step - loss:
0.0296
Epoch 100/350
1/1      0s 24ms/step - loss:
0.0270
Epoch 101/350
1/1      0s 24ms/step - loss:
0.0250
Epoch 102/350
1/1      0s 25ms/step - loss:
0.0231
Epoch 103/350
1/1      0s 27ms/step - loss:
0.0205
Epoch 104/350
1/1      0s 30ms/step - loss:
0.0182
Epoch 105/350
1/1      0s 62ms/step - loss:
0.0167
Epoch 106/350
1/1      0s 30ms/step - loss:
0.0155
Epoch 107/350
1/1      0s 24ms/step - loss:
0.0146
Epoch 108/350
1/1      0s 24ms/step - loss:
0.0146
Epoch 109/350
1/1      0s 25ms/step - loss:
0.0143
Epoch 110/350
1/1      0s 24ms/step - loss:

```

```

0.0135
Epoch 111/350
1/1      0s 25ms/step - loss:
0.0128
Epoch 112/350
1/1      0s 25ms/step - loss:
0.0117
Epoch 113/350
1/1      0s 29ms/step - loss:
0.0104
Epoch 114/350
1/1      0s 26ms/step - loss:
0.0092
Epoch 115/350
1/1      0s 23ms/step - loss:
0.0078
Epoch 116/350
1/1      0s 24ms/step - loss:
0.0069
Epoch 117/350
1/1      0s 24ms/step - loss:
0.0068
Epoch 118/350
1/1      0s 24ms/step - loss:
0.0065
Epoch 119/350
1/1      0s 25ms/step - loss:
0.0060
Epoch 120/350
1/1      0s 27ms/step - loss:
0.0052
Epoch 121/350
1/1      0s 27ms/step - loss:
0.0042
Epoch 122/350
1/1      0s 27ms/step - loss:
0.0035
Epoch 123/350
1/1      0s 27ms/step - loss:
0.0030
Epoch 124/350
1/1      0s 29ms/step - loss:
0.0028
Epoch 125/350
1/1      0s 23ms/step - loss:
0.0029
Epoch 126/350
1/1      0s 25ms/step - loss:

```



```

0.0025
Epoch 127/350
1/1      0s 23ms/step - loss:
0.0020
Epoch 128/350
1/1      0s 24ms/step - loss:
0.0017
Epoch 129/350
1/1      0s 24ms/step - loss:
0.0016
Epoch 130/350
1/1      0s 25ms/step - loss:
0.0015
Epoch 131/350
1/1      0s 25ms/step - loss:
0.0015
Epoch 132/350
1/1      0s 29ms/step - loss:
0.0014
Epoch 133/350
1/1      0s 30ms/step - loss:
0.0013
Epoch 134/350
1/1      0s 24ms/step - loss:
0.0012
Epoch 135/350
1/1      0s 29ms/step - loss:
0.0011
Epoch 136/350
1/1      0s 23ms/step - loss:
0.0010
Epoch 137/350
1/1      0s 24ms/step - loss:
9.7387e-04
Epoch 138/350
1/1      0s 57ms/step - loss:
9.3439e-04
Epoch 139/350
1/1      0s 25ms/step - loss:
8.8802e-04
Epoch 140/350
1/1      0s 25ms/step - loss:
8.5723e-04
Epoch 141/350
1/1      0s 27ms/step - loss:
8.1738e-04
Epoch 142/350
1/1      0s 29ms/step - loss:

```

```

7.7388e-04
Epoch 143/350
1/1      0s 27ms/step - loss:
7.3348e-04
Epoch 144/350
1/1      0s 23ms/step - loss:
6.8968e-04
Epoch 145/350
1/1      0s 24ms/step - loss:
6.5971e-04
Epoch 146/350
1/1      0s 24ms/step - loss:
6.3111e-04
Epoch 147/350
1/1      0s 24ms/step - loss:
6.0502e-04
Epoch 148/350
1/1      0s 25ms/step - loss:
5.8069e-04
Epoch 149/350
1/1      0s 26ms/step - loss:
5.5040e-04
Epoch 150/350
1/1      0s 28ms/step - loss:
5.2099e-04
Epoch 151/350
1/1      0s 25ms/step - loss:
4.8580e-04
Epoch 152/350
1/1      0s 28ms/step - loss:
4.5220e-04
Epoch 153/350
1/1      0s 30ms/step - loss:
4.2559e-04
Epoch 154/350
1/1      0s 22ms/step - loss:
4.0480e-04
Epoch 155/350
1/1      0s 26ms/step - loss:
3.9103e-04
Epoch 156/350
1/1      0s 23ms/step - loss:
3.7568e-04
Epoch 157/350
1/1      0s 24ms/step - loss:
3.6151e-04
Epoch 158/350
1/1      0s 24ms/step - loss:

```

3.4755e-04
 Epoch 159/350
 1/1 0s 25ms/step - loss:
 3.3392e-04
 Epoch 160/350
 1/1 0s 25ms/step - loss:
 3.2296e-04
 Epoch 161/350
 1/1 0s 27ms/step - loss:
 3.1215e-04
 Epoch 162/350
 1/1 0s 29ms/step - loss:
 3.0334e-04
 Epoch 163/350
 1/1 0s 30ms/step - loss:
 2.9398e-04
 Epoch 164/350
 1/1 0s 29ms/step - loss:
 2.8512e-04
 Epoch 165/350
 1/1 0s 22ms/step - loss:
 2.7739e-04
 Epoch 166/350
 1/1 0s 29ms/step - loss:
 2.6957e-04
 Epoch 167/350
 1/1 0s 26ms/step - loss:
 2.6255e-04
 Epoch 168/350
 1/1 0s 23ms/step - loss:
 2.5518e-04
 Epoch 169/350
 1/1 0s 24ms/step - loss:
 2.4873e-04
 Epoch 170/350
 1/1 0s 53ms/step - loss:
 2.4265e-04
 Epoch 171/350
 1/1 0s 27ms/step - loss:
 2.3683e-04
 Epoch 172/350
 1/1 0s 29ms/step - loss:
 2.3114e-04
 Epoch 173/350
 1/1 0s 28ms/step - loss:
 2.2480e-04
 Epoch 174/350
 1/1 0s 28ms/step - loss:

2.1862e-04
 Epoch 175/350
 1/1 0s 25ms/step - loss:
 2.1220e-04
 Epoch 176/350
 1/1 0s 27ms/step - loss:
 2.0650e-04
 Epoch 177/350
 1/1 0s 28ms/step - loss:
 2.0156e-04
 Epoch 178/350
 1/1 0s 26ms/step - loss:
 1.9725e-04
 Epoch 179/350
 1/1 0s 25ms/step - loss:
 1.9325e-04
 Epoch 180/350
 1/1 0s 25ms/step - loss:
 1.8904e-04
 Epoch 181/350
 1/1 0s 25ms/step - loss:
 1.8473e-04
 Epoch 182/350
 1/1 0s 26ms/step - loss:
 1.7999e-04
 Epoch 183/350
 1/1 0s 26ms/step - loss:
 1.7557e-04
 Epoch 184/350
 1/1 0s 26ms/step - loss:
 1.7140e-04
 Epoch 185/350
 1/1 0s 29ms/step - loss:
 1.6763e-04
 Epoch 186/350
 1/1 0s 29ms/step - loss:
 1.6415e-04
 Epoch 187/350
 1/1 0s 27ms/step - loss:
 1.6077e-04
 Epoch 188/350
 1/1 0s 25ms/step - loss:
 1.5751e-04
 Epoch 189/350
 1/1 0s 26ms/step - loss:
 1.5418e-04
 Epoch 190/350
 1/1 0s 26ms/step - loss:

```

1.5096e-04
Epoch 191/350
1/1      0s 25ms/step - loss:
1.4783e-04
Epoch 192/350
1/1      0s 26ms/step - loss:
1.4518e-04
Epoch 193/350
1/1      0s 26ms/step - loss:
1.4279e-04
Epoch 194/350
1/1      0s 26ms/step - loss:
1.4097e-04
Epoch 195/350
1/1      0s 27ms/step - loss:
1.4041e-04
Epoch 196/350
1/1      0s 28ms/step - loss:
1.4259e-04
Epoch 197/350
1/1      0s 25ms/step - loss:
1.5073e-04
Epoch 198/350
1/1      0s 26ms/step - loss:
1.7199e-04
Epoch 199/350
1/1      0s 55ms/step - loss:
2.1987e-04
Epoch 200/350
1/1      0s 29ms/step - loss:
3.1424e-04
Epoch 201/350
1/1      0s 27ms/step - loss:
4.5264e-04
Epoch 202/350
1/1      0s 28ms/step - loss:
5.6060e-04
Epoch 203/350
1/1      0s 28ms/step - loss:
4.6984e-04
Epoch 204/350
1/1      0s 30ms/step - loss:
2.2548e-04
Epoch 205/350
1/1      0s 23ms/step - loss:
1.1445e-04
Epoch 206/350
1/1      0s 24ms/step - loss:

```

2.3891e-04
 Epoch 207/350
 1/1 0s 23ms/step - loss:
 3.4743e-04
 Epoch 208/350
 1/1 0s 24ms/step - loss:
 2.4289e-04
 Epoch 209/350
 1/1 0s 24ms/step - loss:
 1.1049e-04
 Epoch 210/350
 1/1 0s 26ms/step - loss:
 1.5802e-04
 Epoch 211/350
 1/1 0s 26ms/step - loss:
 2.4823e-04
 Epoch 212/350
 1/1 0s 25ms/step - loss:
 1.9490e-04
 Epoch 213/350
 1/1 0s 28ms/step - loss:
 1.0261e-04
 Epoch 214/350
 1/1 0s 29ms/step - loss:
 1.3447e-04
 Epoch 215/350
 1/1 0s 23ms/step - loss:
 1.9387e-04
 Epoch 216/350
 1/1 0s 29ms/step - loss:
 1.5015e-04
 Epoch 217/350
 1/1 0s 32ms/step - loss:
 9.3210e-05
 Epoch 218/350
 1/1 0s 23ms/step - loss:
 1.2235e-04
 Epoch 219/350
 1/1 0s 23ms/step - loss:
 1.5514e-04
 Epoch 220/350
 1/1 0s 24ms/step - loss:
 1.1871e-04
 Epoch 221/350
 1/1 0s 24ms/step - loss:
 8.6771e-05
 Epoch 222/350
 1/1 0s 24ms/step - loss:

```

1.1189e-04
Epoch 223/350
1/1      0s 25ms/step - loss:
1.2811e-04
Epoch 224/350
1/1      0s 25ms/step - loss:
9.8774e-05
Epoch 225/350
1/1      0s 25ms/step - loss:
8.2130e-05
Epoch 226/350
1/1      0s 25ms/step - loss:
1.0136e-04
Epoch 227/350
1/1      0s 24ms/step - loss:
1.0767e-04
Epoch 228/350
1/1      0s 52ms/step - loss:
8.6500e-05
Epoch 229/350
1/1      0s 27ms/step - loss:
7.7741e-05
Epoch 230/350
1/1      0s 29ms/step - loss:
9.0987e-05
Epoch 231/350
1/1      0s 23ms/step - loss:
9.3805e-05
Epoch 232/350
1/1      0s 29ms/step - loss:
7.9175e-05
Epoch 233/350
1/1      0s 27ms/step - loss:
7.3256e-05
Epoch 234/350
1/1      0s 29ms/step - loss:
8.1732e-05
Epoch 235/350
1/1      0s 23ms/step - loss:
8.3801e-05
Epoch 236/350
1/1      0s 25ms/step - loss:
7.4112e-05
Epoch 237/350
1/1      0s 24ms/step - loss:
6.8959e-05
Epoch 238/350
1/1      0s 24ms/step - loss:

```

```

7.3766e-05
Epoch 239/350
1/1      0s 23ms/step - loss:
7.5989e-05
Epoch 240/350
1/1      0s 39ms/step - loss:
6.9969e-05
Epoch 241/350
1/1      0s 36ms/step - loss:
6.5187e-05
Epoch 242/350
1/1      0s 34ms/step - loss:
6.7261e-05
Epoch 243/350
1/1      0s 24ms/step - loss:
6.9495e-05
Epoch 244/350
1/1      0s 24ms/step - loss:
6.6329e-05
Epoch 245/350
1/1      0s 24ms/step - loss:
6.2058e-05
Epoch 246/350
1/1      0s 24ms/step - loss:
6.1978e-05
Epoch 247/350
1/1      0s 26ms/step - loss:
6.3788e-05
Epoch 248/350
1/1      0s 27ms/step - loss:
6.2767e-05
Epoch 249/350
1/1      0s 26ms/step - loss:
5.9526e-05
Epoch 250/350
1/1      0s 28ms/step - loss:
5.7946e-05
Epoch 251/350
1/1      0s 30ms/step - loss:
5.8709e-05
Epoch 252/350
1/1      0s 26ms/step - loss:
5.8979e-05
Epoch 253/350
1/1      0s 23ms/step - loss:
5.7223e-05
Epoch 254/350
1/1      0s 24ms/step - loss:

```


5.5136e-05
 Epoch 255/350
 1/1 0s 25ms/step - loss:
 5.4544e-05
 Epoch 256/350
 1/1 0s 57ms/step - loss:
 5.4897e-05
 Epoch 257/350
 1/1 0s 28ms/step - loss:
 5.4501e-05
 Epoch 258/350
 1/1 0s 25ms/step - loss:
 5.3024e-05
 Epoch 259/350
 1/1 0s 28ms/step - loss:
 5.1649e-05
 Epoch 260/350
 1/1 0s 31ms/step - loss:
 5.1202e-05
 Epoch 261/350
 1/1 0s 26ms/step - loss:
 5.1177e-05
 Epoch 262/350
 1/1 0s 27ms/step - loss:
 5.0658e-05
 Epoch 263/350
 1/1 0s 24ms/step - loss:
 4.9545e-05
 Epoch 264/350
 1/1 0s 23ms/step - loss:
 4.8492e-05
 Epoch 265/350
 1/1 0s 24ms/step - loss:
 4.7964e-05
 Epoch 266/350
 1/1 0s 24ms/step - loss:
 4.7732e-05
 Epoch 267/350
 1/1 0s 25ms/step - loss:
 4.7294e-05
 Epoch 268/350
 1/1 0s 24ms/step - loss:
 4.6491e-05
 Epoch 269/350
 1/1 0s 24ms/step - loss:
 4.5614e-05
 Epoch 270/350
 1/1 0s 25ms/step - loss:

4.4981e-05
 Epoch 271/350
 1/1 0s 25ms/step - loss:
 4.4598e-05
 Epoch 272/350
 1/1 0s 25ms/step - loss:
 4.4228e-05
 Epoch 273/350
 1/1 0s 25ms/step - loss:
 4.3680e-05
 Epoch 274/350
 1/1 0s 25ms/step - loss:
 4.2984e-05
 Epoch 275/350
 1/1 0s 25ms/step - loss:
 4.2309e-05
 Epoch 276/350
 1/1 0s 27ms/step - loss:
 4.1778e-05
 Epoch 277/350
 1/1 0s 29ms/step - loss:
 4.1367e-05
 Epoch 278/350
 1/1 0s 22ms/step - loss:
 4.0957e-05
 Epoch 279/350
 1/1 0s 29ms/step - loss:
 4.0455e-05
 Epoch 280/350
 1/1 0s 30ms/step - loss:
 3.9876e-05
 Epoch 281/350
 1/1 0s 23ms/step - loss:
 3.9293e-05
 Epoch 282/350
 1/1 0s 61ms/step - loss:
 3.8770e-05
 Epoch 283/350
 1/1 0s 23ms/step - loss:
 3.8319e-05
 Epoch 284/350
 1/1 0s 24ms/step - loss:
 3.7900e-05
 Epoch 285/350
 1/1 0s 24ms/step - loss:
 3.7463e-05
 Epoch 286/350
 1/1 0s 24ms/step - loss:

3.6988e-05
 Epoch 287/350
 1/1 0s 26ms/step - loss:
 3.6487e-05
 Epoch 288/350
 1/1 0s 27ms/step - loss:
 3.5992e-05
 Epoch 289/350
 1/1 0s 25ms/step - loss:
 3.5525e-05
 Epoch 290/350
 1/1 0s 28ms/step - loss:
 3.5093e-05
 Epoch 291/350
 1/1 0s 30ms/step - loss:
 3.4683e-05
 Epoch 292/350
 1/1 0s 23ms/step - loss:
 3.4275e-05
 Epoch 293/350
 1/1 0s 24ms/step - loss:
 3.3859e-05
 Epoch 294/350
 1/1 0s 23ms/step - loss:
 3.3430e-05
 Epoch 295/350
 1/1 0s 24ms/step - loss:
 3.2997e-05
 Epoch 296/350
 1/1 0s 24ms/step - loss:
 3.2569e-05
 Epoch 297/350
 1/1 0s 25ms/step - loss:
 3.2154e-05
 Epoch 298/350
 1/1 0s 25ms/step - loss:
 3.1754e-05
 Epoch 299/350
 1/1 0s 25ms/step - loss:
 3.1367e-05
 Epoch 300/350
 1/1 0s 26ms/step - loss:
 3.0989e-05
 Epoch 301/350
 1/1 0s 25ms/step - loss:
 3.0616e-05
 Epoch 302/350
 1/1 0s 27ms/step - loss:

```

3.0245e-05
Epoch 303/350
1/1      0s 29ms/step - loss:
2.9874e-05
Epoch 304/350
1/1      0s 23ms/step - loss:
2.9503e-05
Epoch 305/350
1/1      0s 59ms/step - loss:
2.9135e-05
Epoch 306/350
1/1      0s 31ms/step - loss:
2.8770e-05
Epoch 307/350
1/1      0s 27ms/step - loss:
2.8410e-05
Epoch 308/350
1/1      0s 27ms/step - loss:
2.8056e-05
Epoch 309/350
1/1      0s 23ms/step - loss:
2.7708e-05
Epoch 310/350
1/1      0s 24ms/step - loss:
2.7366e-05
Epoch 311/350
1/1      0s 24ms/step - loss:
2.7029e-05
Epoch 312/350
1/1      0s 24ms/step - loss:
2.6699e-05
Epoch 313/350
1/1      0s 24ms/step - loss:
2.6373e-05
Epoch 314/350
1/1      0s 25ms/step - loss:
2.6052e-05
Epoch 315/350
1/1      0s 25ms/step - loss:
2.5736e-05
Epoch 316/350
1/1      0s 23ms/step - loss:
2.5424e-05
Epoch 317/350
1/1      0s 25ms/step - loss:
2.5117e-05
Epoch 318/350
1/1      0s 25ms/step - loss:

```

2.4816e-05
 Epoch 319/350
 1/1 0s 28ms/step - loss:
 2.4519e-05
 Epoch 320/350
 1/1 0s 27ms/step - loss:
 2.4228e-05
 Epoch 321/350
 1/1 0s 28ms/step - loss:
 2.3945e-05
 Epoch 322/350
 1/1 0s 26ms/step - loss:
 2.3670e-05
 Epoch 323/350
 1/1 0s 26ms/step - loss:
 2.3408e-05
 Epoch 324/350
 1/1 0s 26ms/step - loss:
 2.3164e-05
 Epoch 325/350
 1/1 0s 26ms/step - loss:
 2.2946e-05
 Epoch 326/350
 1/1 0s 27ms/step - loss:
 2.2772e-05
 Epoch 327/350
 1/1 0s 28ms/step - loss:
 2.2670e-05
 Epoch 328/350
 1/1 0s 55ms/step - loss:
 2.2692e-05
 Epoch 329/350
 1/1 0s 24ms/step - loss:
 2.2934e-05
 Epoch 330/350
 1/1 0s 23ms/step - loss:
 2.3566e-05
 Epoch 331/350
 1/1 0s 25ms/step - loss:
 2.4920e-05
 Epoch 332/350
 1/1 0s 25ms/step - loss:
 2.7592e-05
 Epoch 333/350
 1/1 0s 26ms/step - loss:
 3.2755e-05
 Epoch 334/350
 1/1 0s 28ms/step - loss:

4.2513e-05
 Epoch 335/350
 1/1 0s 28ms/step - loss:
 6.0994e-05
 Epoch 336/350
 1/1 0s 26ms/step - loss:
 9.4928e-05
 Epoch 337/350
 1/1 0s 28ms/step - loss:
 1.5613e-04
 Epoch 338/350
 1/1 0s 29ms/step - loss:
 2.5511e-04
 Epoch 339/350
 1/1 0s 26ms/step - loss:
 3.9057e-04
 Epoch 340/350
 1/1 0s 23ms/step - loss:
 5.0818e-04
 Epoch 341/350
 1/1 0s 24ms/step - loss:
 4.9446e-04
 Epoch 342/350
 1/1 0s 24ms/step - loss:
 3.0249e-04
 Epoch 343/350
 1/1 0s 24ms/step - loss:
 7.5887e-05
 Epoch 344/350
 1/1 0s 25ms/step - loss:
 3.0151e-05
 Epoch 345/350
 1/1 0s 43ms/step - loss:
 1.6280e-04
 Epoch 346/350
 1/1 0s 44ms/step - loss:
 2.6155e-04
 Epoch 347/350
 1/1 0s 28ms/step - loss:
 1.8097e-04
 Epoch 348/350
 1/1 0s 30ms/step - loss:
 4.3831e-05
 Epoch 349/350
 1/1 0s 62ms/step - loss:
 3.3211e-05
 Epoch 350/350
 1/1 0s 24ms/step - loss:

1.2593e-04

[11]: <keras.src.callbacks.history.History at 0x141813320>

```
[12]: best_ber = float('inf')

for iteration in range(30):
    print(f"==== Iteration {iteration} =====")

    # symbDPD = symbTx_nn if iteration == 0 else dpd_model.predict(symbTx_nn,
    ↪ verbose=0)
    symbDPD = dpd_model.predict(symbTx_nn, verbose=0)

    y_CPR_1, d= simulate_optical_system(merge_i_q(symbDPD).flatten(),
    ↪ paramSymb, paramPulse,paramIQM,sigL0, paramCh, paramL0, paramFE,
    ↪ paramPD,paramRxPulse,paramDec
    ,paramEDC,paramEq,paramCPR, PA_enable=False,
    ↪ Data_Aided=True)

    ber = perf_calc(symbTx, y_CPR_1, d, M, paramSymb, paramDec)

    if ber < best_ber:
        dpd_model.save_weights("best_model.weights.h5")
        best_ber = ber

    y_CPR_1_nn = split_i_q(y_CPR_1).reshape(-1, seq_length, 2)

    # y = (y_CPR_1 - np.mean(y_CPR_1)) / np.std(y_CPR_1)
    # y_nn = split_i_q(y).reshape(-1, seq_length, 2)

    dpd_model.fit(y_CPR_1_nn, symbDPD, epochs=100, verbose=0) # ILA

    ## PLEASE IGNORE THE BELOW CODE
    # aux_model.fit(symbDPD, y_CPR_1_nn, epochs=200, verbose=0)
    # aux_model.trainable = False
    # dla_cascade.compile(optimizer=tf.keras.optimizers.
    ↪ Adam(learning_rate=1e-3), loss='mse')

    # dla_cascade.fit(symbTx_nn, symbTx_nn, epochs=100, verbose=0)
    # aux_model.trainable = True
    # dla_cascade.compile(optimizer=tf.keras.optimizers.
    ↪ Adam(learning_rate=1e-3), loss='mse')
```

```

===== Iteration 0 =====

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.030379.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.024349.
rde - training stage #1
rde MSE = 0.019012.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 402.195 kHz

SER: 5.111e-04,
BER: 1.278e-04
SNR: 20.113 dB
EVM: 0.973 %
Qfactor: 5.631,
===== Iteration 1 =====

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.034275.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028101.
rde - training stage #1
rde MSE = 0.012457.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 356.911 kHz

SER: 1.889e-04,
BER: 4.722e-05
SNR: 21.668 dB
EVM: 0.688 %
Qfactor: 5.916,
===== Iteration 2 =====

0%|          | 0/1 [00:00<?, ?it/s]

```



```

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.034614.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028458.
rde - training stage #1
rde MSE = 0.010584.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 317.395 kHz

```

```

SER: 1.889e-04,
BER: 4.722e-05
SNR: 22.551 dB
EVM: 0.556 %
Qfactor: 5.916,
===== Iteration 3 =====

```

```

0%|          | 0/1 [00:00<?, ?it/s]

```

```

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.034673.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028513.
rde - training stage #1
rde MSE = 0.010221.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 311.873 kHz

```

```

SER: 1.889e-04,
BER: 4.722e-05
SNR: 22.700 dB
EVM: 0.537 %
Qfactor: 5.916,
===== Iteration 4 =====

```

```

0%|          | 0/1 [00:00<?, ?it/s]

```

```

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...

```

```

da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.034713.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028558.
rde - training stage #1
rde MSE = 0.010124.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 308.682 kHz

SER: 1.556e-04,
BER: 3.889e-05
SNR: 22.782 dB
EVM: 0.527 %
Qfactor: 5.967,
===== Iteration 5 =====

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.034744.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028597.
rde - training stage #1
rde MSE = 0.010123.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 312.486 kHz

SER: 1.778e-04,
BER: 4.444e-05
SNR: 22.823 dB
EVM: 0.522 %
Qfactor: 5.932,
===== Iteration 6 =====

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.034901.

```

```

da-rde pre-convergence training iteration #1
da-rde MSE = 0.028675.
rde - training stage #1
rde MSE = 0.014987.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 305.860 kHz

```

```

SER: 3.778e-04,
BER: 9.444e-05
SNR: 22.016 dB
EVM: 0.643 %
Qfactor: 5.721,
===== Iteration 7 =====

```

```

0%|          | 0/1 [00:00<?, ?it/s]

```

```

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.034663.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028466.
rde - training stage #1
rde MSE = 0.011906.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 282.849 kHz

```

```

SER: 2.222e-04,
BER: 5.556e-05
SNR: 22.833 dB
EVM: 0.529 %
Qfactor: 5.871,
===== Iteration 8 =====

```

```

0%|          | 0/1 [00:00<?, ?it/s]

```

```

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.035006.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028673.
rde - training stage #1

```

```

rde MSE = 0.014980.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 323.961 kHz

SER: 9.000e-04,
BER: 2.250e-04
SNR: 21.289 dB
EVM: 0.759 %
Qfactor: 5.452,
===== Iteration 9 =====

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.034610.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028393.
rde - training stage #1
rde MSE = 0.014059.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 375.933 kHz

SER: 2.944e-03,
BER: 7.361e-04
SNR: 21.366 dB
EVM: 0.744 %
Qfactor: 5.024,
===== Iteration 10 =====

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.034915.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028697.
rde - training stage #1
rde MSE = 0.012870.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]

```

Running BPS carrier phase recovery...

Estimated linewidth: 355.561 kHz

SER: 1.589e-03,

BER: 3.972e-04

SNR: 21.802 dB

EVM: 0.672 %

Qfactor: 5.257,

===== Iteration 11 =====

0%| | 0/1 [00:00<?, ?it/s]

Running CD compensation...

CD filter length: 46 taps, FFT size: 64

Running adaptive equalizer...

da-rde - training stage #0

da-rde pre-convergence training iteration #0

da-rde MSE = 0.035024.

da-rde pre-convergence training iteration #1

da-rde MSE = 0.028805.

rde - training stage #1

rde MSE = 0.012277.

Running frequency offset compensation...

Estimated frequency offset (MHz): [128.24]

Running BPS carrier phase recovery...

Estimated linewidth: 342.921 kHz

SER: 1.067e-03,

BER: 2.667e-04

SNR: 22.086 dB

EVM: 0.630 %

Qfactor: 5.395,

===== Iteration 12 =====

0%| | 0/1 [00:00<?, ?it/s]

Running CD compensation...

CD filter length: 46 taps, FFT size: 64

Running adaptive equalizer...

da-rde - training stage #0

da-rde pre-convergence training iteration #0

da-rde MSE = 0.035073.

da-rde pre-convergence training iteration #1

da-rde MSE = 0.028857.

rde - training stage #1

rde MSE = 0.011778.

Running frequency offset compensation...

Estimated frequency offset (MHz): [128.24]

Running BPS carrier phase recovery...

Estimated linewidth: 315.064 kHz

```

SER: 6.667e-04,
BER: 1.667e-04
SNR: 22.339 dB
EVM: 0.594 %
Qfactor: 5.548,
===== Iteration 13 =====

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.035110.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028897.
rde - training stage #1
rde MSE = 0.011399.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 307.700 kHz

SER: 5.333e-04,
BER: 1.333e-04
SNR: 22.538 dB
EVM: 0.568 %
Qfactor: 5.618,
===== Iteration 14 =====

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.035114.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028905.
rde - training stage #1
rde MSE = 0.011080.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 302.055 kHz

SER: 4.444e-04,
BER: 1.111e-04
SNR: 22.710 dB

```

```

EVM: 0.546 %
Qfactor: 5.673,
===== Iteration 15 =====

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.035111.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028904.
rde - training stage #1
rde MSE = 0.010818.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 297.514 kHz

SER: 3.667e-04,
BER: 9.167e-05
SNR: 22.847 dB
EVM: 0.529 %
Qfactor: 5.730,
===== Iteration 16 =====

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.035111.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028907.
rde - training stage #1
rde MSE = 0.010617.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 285.242 kHz

SER: 3.444e-04,
BER: 8.611e-05
SNR: 22.952 dB
EVM: 0.517 %
Qfactor: 5.748,
===== Iteration 17 =====

```

```

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.035105.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028903.
rde - training stage #1
rde MSE = 0.010443.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 288.617 kHz

SER: 3.333e-04,
BER: 8.333e-05
SNR: 23.036 dB
EVM: 0.507 %
Qfactor: 5.757,
===== Iteration 18 =====

```

```

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.035108.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028910.
rde - training stage #1
rde MSE = 0.010291.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 284.567 kHz

SER: 2.778e-04,
BER: 6.944e-05
SNR: 23.106 dB
EVM: 0.499 %
Qfactor: 5.809,
===== Iteration 19 =====

```

```

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...

```



```

CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.035116.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028918.
rde - training stage #1
rde MSE = 0.010188.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 279.658 kHz

```

```

SER: 2.778e-04,
BER: 6.944e-05
SNR: 23.153 dB
EVM: 0.493 %
Qfactor: 5.809,
===== Iteration 20 =====

```

```

0%|          | 0/1 [00:00<?, ?it/s]

```

```

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.035134.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028937.
rde - training stage #1
rde MSE = 0.010114.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 278.984 kHz

```

```

SER: 2.778e-04,
BER: 6.944e-05
SNR: 23.185 dB
EVM: 0.490 %
Qfactor: 5.809,
===== Iteration 21 =====

```

```

0%|          | 0/1 [00:00<?, ?it/s]

```

```

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0

```

```

da-rde pre-convergence training iteration #0
da-rde MSE = 0.035147.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028951.
rde - training stage #1
rde MSE = 0.010054.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 272.479 kHz

```

```

SER: 2.667e-04,
BER: 6.667e-05
SNR: 23.207 dB
EVM: 0.487 %
Qfactor: 5.821,
===== Iteration 22 =====

```

```

0%|          | 0/1 [00:00<?, ?it/s]

```

```

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.035158.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028962.
rde - training stage #1
rde MSE = 0.010011.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 271.498 kHz

```

```

SER: 2.667e-04,
BER: 6.667e-05
SNR: 23.220 dB
EVM: 0.485 %
Qfactor: 5.821,
===== Iteration 23 =====

```

```

0%|          | 0/1 [00:00<?, ?it/s]

```

```

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.035166.
da-rde pre-convergence training iteration #1

```

```

da-rde MSE = 0.028971.
rde - training stage #1
rde MSE = 0.009978.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 272.848 kHz

SER: 2.667e-04,
BER: 6.667e-05
SNR: 23.230 dB
EVM: 0.484 %
Qfactor: 5.821,
===== Iteration 24 =====

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.035175.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028979.
rde - training stage #1
rde MSE = 0.009951.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 271.007 kHz

SER: 2.778e-04,
BER: 6.944e-05
SNR: 23.239 dB
EVM: 0.483 %
Qfactor: 5.809,
===== Iteration 25 =====

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.035176.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028980.
rde - training stage #1
rde MSE = 0.009935.

```

```

Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 272.357 kHz

SER: 2.667e-04,
BER: 6.667e-05
SNR: 23.246 dB
EVM: 0.482 %
Qfactor: 5.821,
===== Iteration 26 =====

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.035162.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028966.
rde - training stage #1
rde MSE = 0.009922.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 274.198 kHz

SER: 2.667e-04,
BER: 6.667e-05
SNR: 23.254 dB
EVM: 0.481 %
Qfactor: 5.821,
===== Iteration 27 =====

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.035158.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028962.
rde - training stage #1
rde MSE = 0.009915.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...

```

Estimated linewidth: 271.007 kHz

SER: 2.667e-04,

BER: 6.667e-05

SNR: 23.258 dB

EVM: 0.481 %

Qfactor: 5.821,

===== Iteration 28 =====

0%| | 0/1 [00:00<?, ?it/s]

Running CD compensation...

CD filter length: 46 taps, FFT size: 64

Running adaptive equalizer...

da-rde - training stage #0

da-rde pre-convergence training iteration #0

da-rde MSE = 0.035165.

da-rde pre-convergence training iteration #1

da-rde MSE = 0.028968.

rde - training stage #1

rde MSE = 0.009918.

Running frequency offset compensation...

Estimated frequency offset (MHz): [128.24]

Running BPS carrier phase recovery...

Estimated linewidth: 272.357 kHz

SER: 2.444e-04,

BER: 6.111e-05

SNR: 23.258 dB

EVM: 0.481 %

Qfactor: 5.845,

===== Iteration 29 =====

0%| | 0/1 [00:00<?, ?it/s]

Running CD compensation...

CD filter length: 46 taps, FFT size: 64

Running adaptive equalizer...

da-rde - training stage #0

da-rde pre-convergence training iteration #0

da-rde MSE = 0.035174.

da-rde pre-convergence training iteration #1

da-rde MSE = 0.028977.

rde - training stage #1

rde MSE = 0.009922.

Running frequency offset compensation...

Estimated frequency offset (MHz): [128.24]

Running BPS carrier phase recovery...

Estimated linewidth: 271.130 kHz

SER: 2.444e-04,

```

BER: 6.111e-05
SNR: 23.256 dB
EVM: 0.481 %
Qfactor: 5.845,

```

```

[13]: def optical_chain_w_dpd(symbTx, DPD_ON=False, Data_Aided=False):
        symbTx_nn = split_i_q(symbTx)
        symbTx_nn = symbTx_nn[:reshape_len*seq_length].reshape(-1, seq_length, 2)
        symbDPD = dpd_model.predict(symbTx_nn, verbose=0) if DPD_ON==True else
        ↪symbTx_nn
        y_CPR_1, d= simulate_optical_system(merge_i_q(symbDPD).flatten(),
        ↪paramSymb, paramPulse,paramIQM,sigLO, paramCh, paramLO, paramFE,
        ↪paramPD,paramRxPulse,paramDec
                                ,paramEDC,paramEq,paramCPR,
        ↪Data_Aided=Data_Aided)
        return y_CPR_1,d

```

0.2 Performance Evaluation

```

[14]: def test(new_seed):
        dpd_model.load_weights("best_model.weights.h5")
        paramSymb.seed = new_seed
        symbTx = symbolSource(paramSymb)
        y_CPR_1_dpd, d_dpd = optical_chain_w_dpd(symbTx, DPD_ON=True,
        ↪Data_Aided=True)
        y_CPR_1, d = optical_chain_w_dpd(symbTx, DPD_ON=False, Data_Aided=True)

        print("==== Perf Without DPD (Data-Blind RX) =====")
        perf_calc(symbTx, y_CPR_1, d, M, paramSymb, paramDec)

        print("==== Perf With DPD (Data-Blind RX): =====")
        perf_calc(symbTx, y_CPR_1_dpd, d_dpd, M, paramSymb, paramDec)

test(444)

```

```

0%|          | 0/1 [00:00<?, ?it/s]

```

```

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.034713.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.028558.
rde - training stage #1

```

```

rde MSE = 0.010124.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 308.682 kHz

0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation...
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer...
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.030226.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.024184.
rde - training stage #1
rde MSE = 0.018600.
Running frequency offset compensation...
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery...
Estimated linewidth: 396.550 kHz

===== Perf Without DPD (Data-Blind RX) =====
SER: 4.444e-04,
BER: 1.111e-04
SNR: 20.272 dB
EVM: 0.938 %
Qfactor: 5.673,
===== Perf With DPD (Data-Blind RX): =====
SER: 1.556e-04,
BER: 3.889e-05
SNR: 22.782 dB
EVM: 0.527 %
Qfactor: 5.967,

```